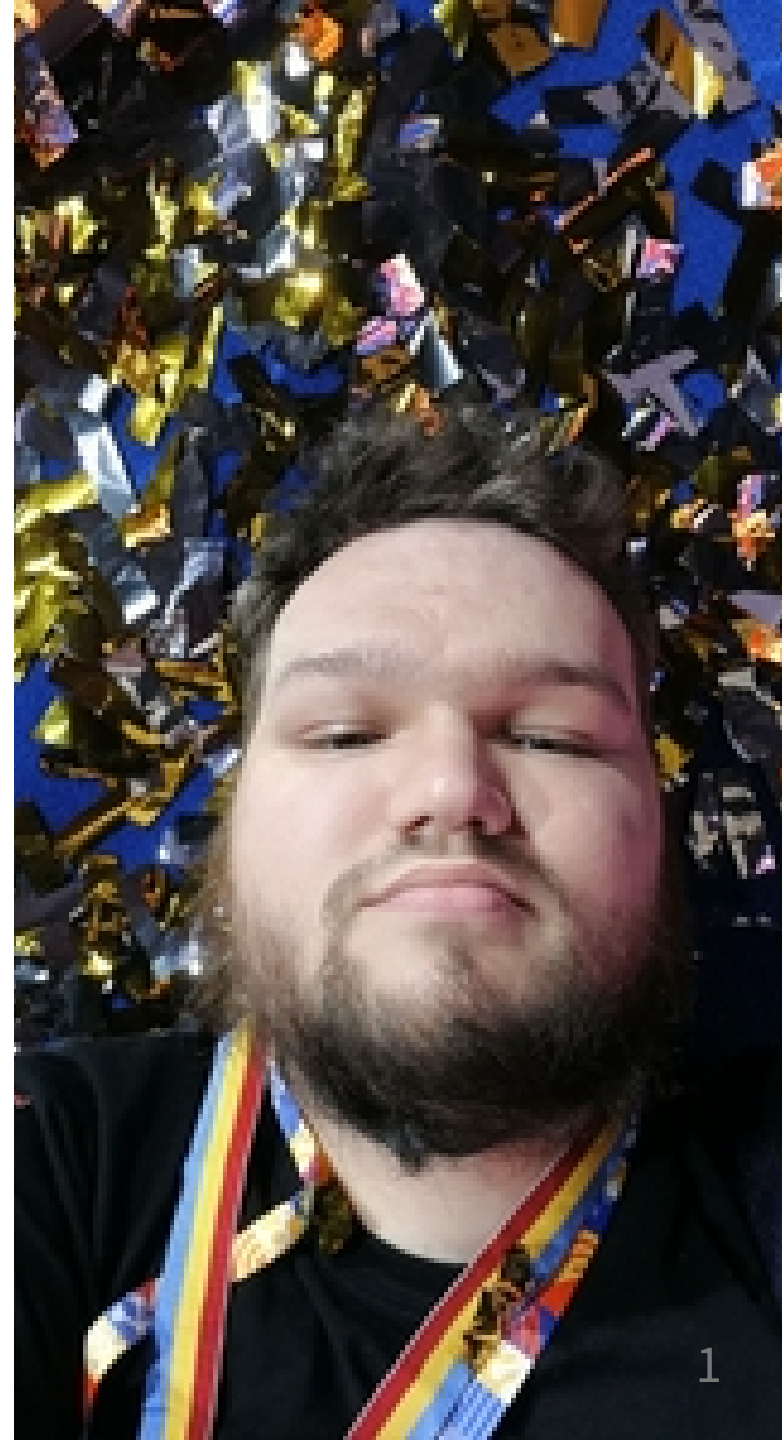


二分查找

Stop learning useless algorithms, go and solve some problems, learn how to use binary search.

— Um_nik



目录

- 二分查找的原理和实现
- `std::lower_bound()` 和 `std::upper_bound()`
- 二分答案

Part 1

二分查找的原理和实现

问题一

有 N 个球排成一行，从左到右从 1 到 N 编号，每个球是黑色或白色。第 1 个球是白色，第 N 个球是黑色，其他球颜色未知。

你可询问某个球的颜色。

你要找出一个整数 i 满足第 i 个球是白色而第 $i + 1$ 个球是黑色。

给出一个询问次数最少的方案。



一定有一对相邻的白球和黑球吗？

询问次数的下界

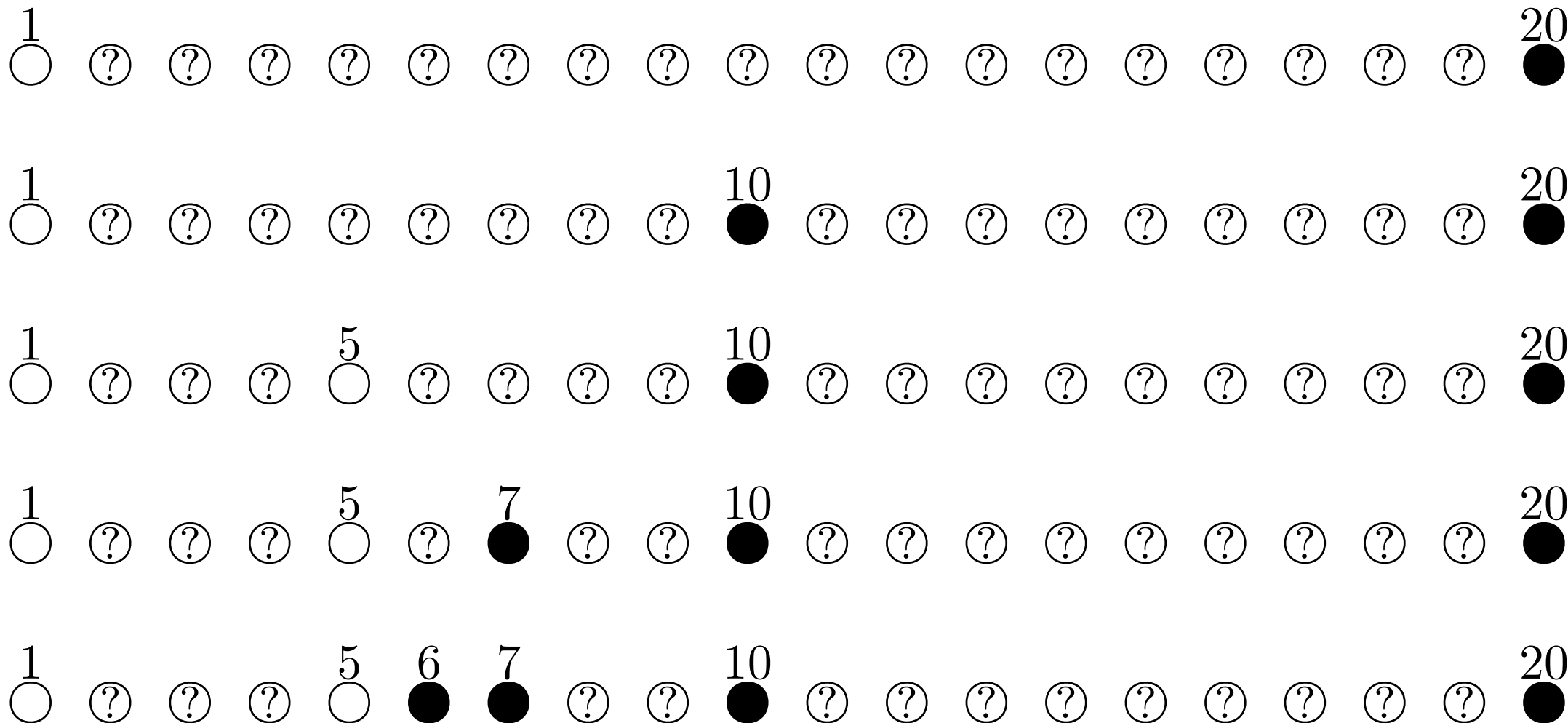
询问次数不可能少于 $\lceil \log_2(N - 1) \rceil$ 。

解释：答案有 $N - 1$ 种可能。每次询问的结果有 2 种可能， k 次询问的结果有 2^k 种可能。必须要有 $2^k \geq N - 1$ 才能凭询问结果**区别** $N - 1$ 种情况。

这个解释逻辑严密吗？

解法

这个问题可用^{binary search}**二分查找**算法来解决。想法如下图所示



伪代码

binary_search(N):

$w \leftarrow 1$

$b \leftarrow N$

while $w + 1 < b$ **do**

$m \leftarrow \lfloor (w + b) / 2 \rfloor$

// 这里 $w < m < b$

if 第 m 个球是白色 **then**

$w \leftarrow m$

else

$b \leftarrow m$

return w

二分查找的询问次数

是 $\lceil \log_2(N - 1) \rceil$ 吗?

模板

```
template<typename F>
long long binary_search(F check, long long ok, long long ng) {
    while (abs(ok - ng) > 1) {
        long long x = (ok + ng) / 2;
        if (check(x))
            ok = x;
        else
            ng = x;
    }
    return ok;
}
```

这里，ng 意思是 no good。

问题二

有 N 个球排成一行，从左到右从 1 到 N 编号，每个球是黑色或白色。第 N 个球是黑色，其他球颜色未知。球的颜色满足：

- 若第 i 个球是白色，则第 1 到第 $i - 1$ 个球全是白色。

你可询问某个球的颜色。你要找出第一个黑球，即找出一个最小的整数 i 满足第 i 个球是黑色。

这是问题一的特例：假想还有第 0 个球，是白色的。

二分查找的第二种写法

binary_search_2(N):

$w \leftarrow 1$ // 第 w 个球颜色未知而前 $w - 1$ 个球都是白色

$b \leftarrow N$ // 第 b 个球是黑色

while $w < b$ **do**

$m \leftarrow \lfloor (w + b) / 2 \rfloor$ // 这里 $w \leq m < b$

if 第 m 个球是黑色 **then**

$b \leftarrow m$

else

$w \leftarrow m + 1$

return b

二分查找的第三种写法

binary_search_3(N):

```
     $w \leftarrow 1$                                 // 第  $w - 1$  个球是白色
     $b \leftarrow N - 1$                           // 第  $b + 1$  个球是黑色
    while  $w \leq b$  do                          // 第  $w$  到  $b$  个球的颜色都未知
         $m \leftarrow \lfloor (w + b) / 2 \rfloor$       // 这里  $w \leq m \leq b$ 
        if 第  $m$  个球是黑色 then
             $b \leftarrow m - 1$ 
        else
             $w \leftarrow m + 1$ 
    return  $b + 1$ 
```

Tips

- 不论哪一种写法，都要**确保每次询问过后答案的范围减小**，不然就可能死循环。
- 第二种写法，一定要写 $m \leftarrow \lfloor w + b \rfloor$ ，不能写 $m \leftarrow \lceil w + b \rceil$ 。因为在 $w < b$ 的情况下 $\lceil w + b \rceil$ 可能等于 b 。
- 妙的是， $\lfloor w + b \rfloor$ 可以用 `(w + b) >> 1` 来表达，即使 $w + b$ 是负数也成立。

binary_search_2(N):

```
w ← 1      // 第 w 个球颜色未知而前 w - 1 个球都是白色
b ← N      // 第 b 个球是黑色
while w < b do
    m ←  $\lfloor (w + b) / 2 \rfloor$       // 这里  $w \leq m < b$ 
    if 第 m 个球是黑色 then
        b ← m
    else
        w ← m + 1
return b
```

Tips

- 理解二分查找的原理。
- 掌握**一种**二分查找的写法。

我推荐第一种写法。

Part 2

`std::lower_bound()` 和 `std::upper_bound()`

范围

在 C++ 里用两个指针或迭代器（比如 beg 和 end）表示一个**范围**（序列里的一段），规则是**左闭右开**，我们以符号 [beg, end) 表示。

std::lower_bound

设 $[b, e)$ 是元素从小到大排好序的范围。lower_bound(b, e, x) 返回指向 $[b, e)$ 内第一个 **不小于** x 的元素的指针（或迭代器），若不存在这样的元素则返回 e 。

用二分查找来实现 lower_bound()

```
lower_bound(int* b, int *e, int x) {  
    while (b != e) {  
        int *m = b + (e - b) / 2;  
        if (*m < x)  
            b = m + 1;  
        else // !(*m < x)  
            e = m;  
    }  
    return e;  
}
```

std::upper_bound

设 $[b, e)$ 是元素从小到大排好序的范围。upper_bound(b, e, x) 返回指向 $[b, e)$ 内第一个 **大于** x 的元素的指针（或迭代器），若不存在这样的元素则返回 e。

用二分查找来实现 upper_bound()

```
lower_bound(int* b, int *e, int x) {  
    while (b != e) {  
        int *m = b + (e - b) / 2;  
        if (x < *m)  
            e = m;  
        else // !(x < *m)  
            b = m + 1;  
    }  
    return e;  
}
```

C++ 里默认的比较函数是小于号

`lower_bound(b, e, v)` 返回指向 `[b, e)` 里第一个满足 “**x 不小于 v**” 的元素 `x` 指针。

“`x 不小于 v`” 用小于号来表达就是 `! xiaoyu(x, v)`

`upper_bound(b, e, v)` 返回指向 `[b, e)` 里第一个满足 “**v 小于 x**” 的元素 `x` 的指针。

“`v 小于 x`” 用小于号来表达就是 `xiaoyu(v, x)`。

自定义的比较函数也得是某种小于号

设元素类型是 T，查找的值的类型是 U（没错，两者可以不同）

对于 `lower_bound(b, e, v, cmp)`

cmp 的格式得是 `bool cmp(T element, U value)`

对于 `upper_bound(b, e, v, cmp)`

cmp 的格式得是 `bool cmp(U value, T element)`

Tips

- **一定要熟练使用** `lower_bound()` 和 `upper_bound()`
- 能用 `lower_bound()` 或 `upper_bound()` 时最好不要手写二分查找。
- `lower_bound` 的比较函数和 `upper_bound` 的比较函数格式不同。

朴素的二分查找（查询一个有序的序列中第一个大于/小于某个值的元素）
通常用在解题的某个步骤。

掌握二分查找就能更顺利地解题。

例题：相交的区间

给你数轴上的 N 个区间。第 i 个区间是 $[\ell_i, r_i]$ 。有多少个数对 (i, j) ($1 \leq i < j \leq N$) 满足第 i 个区间和第 j 个区间相交？

限制

- $2 \leq N \leq 5 \times 10^5$
- $0 \leq \ell_i < r_i \leq 10^9$

样例

输入

输出

解释

3
1 5
7 8
3 7

2

$[1, 5]$ 和 $[3, 7]$ 相交， $[7, 8]$ 和 $[3, 7]$ 相交。

思考一下（5 分钟）

思路

这题有多种解法。我们介绍一个简便思路。

两个区间相交的情况有点多。(其实也不多)

考虑不相交的区间有多少对。

对一个区间 $[\ell, r]$ ，计算完全在它左边的区间有多少个。

即有多少个 r_i 小于 ℓ 。

代码

```
void solve() {
    int n; cin >> n;
    vector<int> l(n), r(n);
    for (int i = 0; i < n; i++)
        cin >> l[i] >> r[i];
    sort(r.begin(), r.end());

    long long ans = 0;
    for (int i = 0; i < n; i++) {
        auto it = lower_bound(r.begin(), r.end(), l[i]);
        ans += it - r.begin();
    }
    cout << (long long) n * (n - 1) / 2 - ans;
}
```

例题：超速检测

有一条南北方向的道路，长度是 L 。

这个周末，道路上预计出现 N 辆车，其中第 i 辆车从主干道上距离最南端 D_i 的位置驶入，以 v_i 的初速度和 A_i 的加速度做匀加速运动向北行驶。我们只考虑从南向北的车辆，故 $v_i > 0$ ，但 A_i 可正可负，也可以为零。当车辆行驶到道路最北端或速度降为零时，我们认为该车驶离道路。

道路上设置了 M 个测速仪，其中第 j 个测速仪位于距离最南端 P_j 的位置，每个测速仪可以设置开启或关闭。当某辆车经过某个开启的测速仪时，若这辆车的瞬时速度**超过了**道路限速 V ，那么这辆车就会被判定为超速。注意当车辆驶入与驶出道路时，如果在对应位置有一个开启的测速仪，这个测速仪也会对这辆车进行测速。

如果所有测速仪都是开启的，那么这 N 辆车中会有多少辆车被判定为超速？

为了节能，交通管理部门想关闭一部分测速仪。然而，他们不希望漏掉超速的车，也就是说，当 N 辆车里的某辆车在所有测速仪都开启时被判定为超速，他们希望在关闭一部分测速仪以后它依然被判定为超速。

在这样的条件下最多可以关闭多少测速仪？

一个输入文件里 包含 T 组数据。

限制

- $1 \leq T \leq 20$;
- $1 \leq N, M \leq 10^5$, $1 \leq L \leq 10^6$, $1 \leq V \leq 10^3$;
- $0 \leq D_i < L$, $1 \leq v_i \leq 10^3$, $|A_i| \leq 10^3$;
- $0 \leq P_1 < P_2 < \dots < P_m \leq L$ 。

思考一下（5 分钟）

这题一点都不难。

思路

- 算出每辆车在哪一段路超速。
- 找出这一段路上的第一个和最后一个测速仪。（可以用**二分查找**）
- 每一辆会被检测到超速的车都对应一个会检测到它超速的测速仪的范围。
- 变成上一讲提到的**区间选点问题**。

代码

```
void solve() {
    vector<pair<int, int>> s; // 区间
    for (int i = 0; i < n; i++) {
        if (d[i] > p[m - 1] || (a[i] <= 0 && v[i] <= V))
            continue; // 跳过
        if (a[i] == 0 || a[i] > 0 && v[i] > V) { // 出发时就超速
            int j = (int) (lower_bound(p.begin(), p.end(), d[i]) - p.begin());
            s.push_back({j, m - 1});
        } else if (a[i] > 0) {
            // x: 开始超速时车的位置
            long long x = d[i] + (1LL * V * V - 1LL * v[i] * v[i]) / (2 * a[i]) + 1;
            auto j = lower_bound(p.begin(), p.end(), x) - p.begin();
            if (j != m) {
                s.push_back({j, m - 1});
            }
        } else { // a[i] < 0
            // 计算车速降到 V 时车的位置
            long long x = d[i] + (1LL * V * V - 1LL * v[i] * v[i] + 1) / (2 * a[i]);
            auto it = lower_bound(p.begin(), p.end(), d[i]);
            auto it2 = upper_bound(p.begin(), p.end(), x);
            if (it != it2) {
                s.push_back({it - p.begin(), it2 - p.begin() - 1});
            }
        }
    }
}

cout << s.size() << ' ' << m - work(s) << '\n'; // work 是区间选点问题
}
```

```
bool cmp(pair<int, int> a, pair<int, int> b) {  
    return a.second < b.second;  
}  
  
int work(vector<pair<int, int>> &s) { // 选最少的点覆盖全部区间  
    sort(s.begin(), s.end(), cmp);  
    int last = -1;  
    int cnt = 0;  
    for (auto p : s) {  
        if (p.first > last) {  
            last = p.second;  
            cnt++;  
        }  
    }  
    return cnt;  
}
```

Part 3

二分答案

许多问题都形如「求使得命题 $P(n)$ 成立的最大（或最小）的整数 n 」。

通常有三种情况

- $P(n)$ 成立当且仅当 $n \geq x$
- $P(n)$ 成立当且仅当 $n \leq x$
- $P(n)$ 成立当且仅当 $\ell \leq n \leq r$

这类问题，如果我们

- 对给定的整数 n ，能判断 $P(n)$ 是否成立，并且
- 知道两个整数 a, b 满足 $P(a)$ 成立且 $P(b)$ 不成立

就能用二分查找来解决。

我们把这种方法称为**二分答案**。

例题：项目计划

某公司有 N 个部门，第 i 个部门有 A_i 个员工 ($1 \leq i \leq N$)。没有一个员工属于多个部门。

公司计划做一些跨部门项目。每个项目需要 K 个来自 K 个不同部门的人。一个员工不能参与多个项目。

最多可以做多少个跨部门项目？

限制

- $1 \leq K \leq N \leq 2 \times 10^5$
- $1 \leq A_i \leq 10^{12}$

给定正整数 x ，如何判断能否做 x 个项目？

设想有 x 个房间。若一个部门有不少于 x 个人，拉来 x 个人，每个房间放一个人。

若人还不够，对于人数不到 x 的部门，一个部门的人全拉来，从第一个房间开始，每一个房间放一个人；下一个部门的人全拉来，从下一个房间开始接着放，第 x 个房间放好后，再回到第 1 个房间，依次类推。

这样安排，人都能用上。

代码模板

以后的代码都采用模板

```
template<typename F>
long long binary_search(F check, long long ok, long long ng) {
    while (abs(ok - ng) > 1) {
        long long x = (ok + ng) / 2;
        if (check(x))
            ok = x;
        else
            ng = x;
    }
    return ok;
}
```

我们只写 check 函数，确定 ok 和 ng 这两个参数。调用 binary_search，

代码

```
int main() {
    int n, k;
    cin >> n >> k;
    vector<long long> a(n);
    for (int i = 0; i < n; i++)
        cin >> a[i];

    auto check = [&](long long x) {
        long long sum = 0;
        for (long long y : a)
            sum += min(y, x);
        return x * k <= sum;
    };

    long long l = 1, r = 3e17 / k;
    cout << binary_search(check, l, r) << '\n';
}
```

例题：花束

有 R 支红花， B 支蓝花。可以制作两种花束。

- 用 x 支红花和 1 支蓝花做一个花束。
- 用 1 支红花和 y 支蓝花做一个花束。

最多可以制作多少个花束？不需要把花用完。

限制

- $1 \leq R, B \leq 10^{18}$
- $2 \leq x, y \leq 10^9$

如何判断能否做 K 个花束?

我来推式子

设第一种花束做 m 个，第二种花束做 n 个，那么需要 $mx + n$ 支红花和 $m + ny$ 支蓝花。我们需要判断在下列约束下 $m + n = K$ 能否成立。

- $mx + n \leq R$
- $m + ny \leq B$
- $m \geq 0, n \geq 0, m, n$ 是整数

把 $mx + n \leq R$ 写成 $m(x - 1) + (m + n) \leq R$ ，即 $m(x - 1) + k \leq R$ 。

所以 $m \leq (R - K)/(x - 1)$ 。

同理可得 $n \leq (B - K)/(y - 1)$ 。

于是可知，只要判断是否有 $\lfloor \frac{R-K}{x-1} \rfloor + \lfloor \frac{B-K}{y-1} \rfloor \geq K$ 。

其实上面的推导是在知道结果以后写出来的

要得到 $\lfloor \frac{R-K}{x-1} \rfloor + \lfloor \frac{B-K}{y-1} \rfloor \geq K$ 这个条件，更自然的想法是

- 要做 K 个花束，拿来 K 个盒子，每个盒子里先放一支红花和一只蓝花。
剩下的红花只有做第一种花束才用得着，剩下的蓝花只有做第二种花束才用得着。

代码

```
long long R, B;
int x, y;
bool check(long long k) {
    return (R - k) / (x - 1) + (B - k) / (y - 1) >= k;
}

cout << binary_search(check, 0, min(R, B)) << '\n';
```

例题：平均数和中位数

我们有 N 张卡片。第 i 张卡片 ($1 \leq i \leq N$) 上写着一个整数 A_i 。

小明将从这 N 张卡片中选一些。但是，对于每个 $i = 1, 2, \dots, N - 1$ ，第 i 张卡片和第 $i + 1$ 张卡片至少要选一张。

求下述两值。

- 选出的卡片上写的数的平均值可能达到的最大值。
- 选出的卡片上写的数的中位数可能达到的最大值。

这里， n 个数的中位数的定义是，把 n 个数从小到大排列后的第 $\lceil \frac{n}{2} \rceil$ 个数。

限制

- $2 \leq N \leq 10^5$
- $1 \leq A_i \leq 10^9$

思考一下（10 分钟）

二分答案

问题一：固定一个**实数** X ，能否从 A_1 到 A_N 中（在满足题目的约束条件的前提下）选一些数使得所选数的平均值 $\geq X$ ？

问题二：固定一个**整数** Y ，能否从 A_1 到 A_N 中（在满足题目的约束条件的前提下）选一些数使得所选数的中位数 $\geq Y$ ？

两个经典的转化

$A_1, \dots, A_N$ 的平均值 $\geq X$ 相当于 $\sum_{i=1}^N (A_i - X) \geq 0$ 。

$A_1, \dots, A_N$ (按本题定义) 的中位数 $\geq X$ 相当于 $A_1, \dots, A_N$ 中 $\geq X$ 的数**超过**一半。

考虑序列 $B = (A_i - X)$ ，问题一变成

- 能否从序列 B 中选一些数使得所选数之和 ≥ 0 ?

问题二变成

- 能否从序列 A 中选一些数使得所选的数中 $\geq Y$ 的超过一半。

这两个新问题都可以用 DP 来解决。

怎么做?

两个 DP

- 能否从序列 $B = (A_i - X)$ 中选一些数使得所选数之和 ≥ 0 ?

$f[i][1/0]$: 从前 $B_1, \dots, B_i$ 中选一些, B_i 个数选/不选, 所选数之和的最大值。

- 能否从序列 A 中选一些数使得所选的数中 $\geq Y$ 的超过一半。

$g[i][1/0]$: 从 $A_1, \dots, A_i$ 中选一些, A_i 选/不选, 所选数中「 $\geq Y$ 的数的个数减去 $< Y$ 的数的个数」的最大值。

这些套路见过就会了, 一点都不难。

二分答案时如何处理答案是浮点数的情况

每次 check，答案的范围缩小一半。

多 check 几次把答案的范围逼得足够小。

```

double f[maxn][2];
int g[maxn][2];

// ... 把缺少的代码补上
auto check_avg = [&](double x) {
    f[0][0] = 0;
    f[0][1] = b[0];
    for (int i = 1; i < n; i++) {
        f[i][0] = f[i - 1][1];
        f[i][1] = b[i] + max(f[i - 1][0], f[i - 1][1]);
    }
    return max(f[n - 1][0], f[n - 1][1]) >= 0;
};

double l = 1, r = 1e9;
for (int t = 0; t < 100; t++) {
    double mid = (l + r) / 2;
    if (check_avg(mid))
        l = mid;
    else
        r = mid;
}
cout << l << '\n';

```

```

auto check_median = [&](int x) {
    vector<int> b(n);
    for (int i = 0; i < n; i++)
        if (a[i] >= x) b[i] = 1;
        else b[i] = -1;
    g[0][0] = 0;
    g[0][1] = b[0];
    for (int i = 1; i < n; i++) {
        g[i][0] = g[i - 1][1];
        g[i][1] = b[i] + max(g[i - 1][0], g[i - 1][1]);
    }
    return max(g[n - 1][0], g[n - 1][1]) > 0;
};

cout << binary_search(check_median, 1, 1e9 + 1) << '\n';

```

代码

```
void solve() {
    int n, m; long long k; cin >> n >> m >> k;
    vector<long long> a(n);
    for (int i = 0; i < n; i++) { cin >> a[i]; k -= a[i]; }
    if (m == n) {
        for (int i = 0; i < n; i++) cout << 0 << ' ';
        return;
    }
    auto b = a; sort(b.begin(), b.end());
    vector<long long> s(n + 1); //前缀和
    for (int i = 0; i < n; i++)
        s[i + 1] = s[i] + b[i];

    for (long long x : a) {
        auto check = [&](long long t) {
            int i = upper_bound(b.begin(), b.end(), x + t) - b.begin();
            if (n - i >= m) return false;
            int cnt = m - (n - i);
            long long have;
            if (b[n - m - 1] >= x) have = s[i] - s[i - cnt];
            else have = s[i] - s[i - cnt - 1] - x;
            return have + k - t < cnt * (x + t + 1);
        };
        long long t = binary_search(check, k + 1, -1);
        if (t == k + 1) t = -1;
        cout << t << ' ';
    }
}
```

例题：扫地机器人 ★

数轴上有 n 个扫地机器人和 m 个垃圾， i 号机器人的初始坐标为 a_i ，第 i 个垃圾的坐标为 b_i 。

扫地机器人每秒的移动距离为 1，分为三个种类：

- 1 类机器人只能向左移动，也就是说如果现在有一个 1 类机器人位于坐标 x ，那么 1 秒后它位于坐标 $x - 1$ ；
- 2 类机器人只能向右移动，也就是说如果现在有一个 2 类机器人位于坐标 x ，那么 1 秒后它位于坐标 $x + 1$ ；
- 3 类机器人可以自己选择向左或向右移动，随时可以改变方向且改变方向不花时间，也就是说如果现在有一个 3 类机器人位于坐标 x ，那么 1 秒后它可能位于坐标 $x - 1$ 或坐标 $x + 1$ 。

一个垃圾被任意一个扫地机器人经过时会被清理，清理垃圾不花时间。多个扫地机器人可以同时位于一个坐标。

求清理所有垃圾最少需要花多少秒。数据保证至少有一个 3 类机器人。

限制

- $1 \leq n \leq 10^5$
- $1 \leq m \leq 2 \times 10^5$
- $1 \leq a_i, b_i \leq 10^{18}$

思考一下（10 分钟）

二分答案

固定非负整数 L ，判断花 L 秒能否清理完所有垃圾。

1 类机器人和 2 类机器人的活动范围是确定的。

问题化为

- 3 类机器人能否在 L 秒内清理完剩余的垃圾？

提示 1

可以加一个限制

- 不要让两个 3 类机器人迎面相遇。

为什么？

提示 2

- 考虑初始位置最靠左的那个（3 类）机器人，如果它左边有垃圾，一定由它负责；否则它就一直向右走。
- 在它左边有垃圾的情况下，它是先向左走还是先向右走好？
- 这要看怎样走覆盖的范围更大。

代码

```
void solve() {
    int n, m;
    vector<long long> a(n), b(m);
    vector<int> type(n);
    // 输入 ...
    vector<long long> p[4];
    for (int i = 0; i < n; i++)
        p[type[i]].push_back(a[i]);
    for (int i = 1; i <= 3; i++)
        sort(p[i].begin(), p[i].end());
    // ... 对 b 排序并去重
    m = (int) b.size();
    // ... check
    cout << binary_search(check, 2e18, -1) << '\n';
}
```

```
auto check = [&](ll x) -> bool {
    vector<int> vis(m);
    int i = 0;
    for (int y : p[1]) {
        // y - x <= b[i] <= y
        for (; i < m && b[i] <= y; i++)
            if (b[i] >= y - x)
                vis[i] = 1;
    }
    i = 0;
    for (int y : p[2]) {
        // y <= b[i] <= y + x
        for (; i < m && b[i] <= y + x; i++)
            if (b[i] >= y) vis[i] = 1;
    }
    i = 0;
    for (int y : p[3]) {
        while (i < m && vis[i]) i++;
        if (i < m) {
            long long len; // y往右最多能够到多远
            // 计算len
            if (y <= b[i]) len = x;
            else {
                if (b[i] + x < y) return false;
                // option 1: 先往右走, 再往左走
                // option 2: 先往左走, 再往右走
                len = max((x - (y - b[i])) / 2, x - 2 * (y - b[i]));
                i++;
            }
            while (i < m && b[i] <= y + len) i++;
        }
    }
    for (; i < m; i++)
        if (!vis[i]) return false;
    return true;
};
```

例题：种树 ★

你要在一片森林内的地块上种树，并养护至树木长到指定的高度。

森林的地图有 n 片地块，其中 1 号地块连接森林的入口。共有 $n - 1$ 条道路连接这些地块，使得每片地块都能通过道路互相到达。最开始，每片地块上都没有树木。

你的目标是：在每片地块上均种植一棵树木，并使得 i 号地块上的树的高度生长到不低于 a_i 米。

你每天可以选择一个未种树且**与某个已种树的地块直接邻接（即通过单条道路相连）**的地块，种一棵高度为 0 米的树。如果所有地块均已种过树，则你当天不进行任何操作。特别地，第 1 天你只能在 1 号空地种树。

一棵树，从种下的当天开始，每天都会生长一定的高度。在第 x 天， i 号地块上的树会长高 $\max(b_i + x \times c_i, 1)$ 米。注意这里的 x 是从整个任务的第一天，而非种下这棵树的第一天开始计算。

最少需要多少天能够完成你的任务？

限制

- $1 \leq n \leq 10^5$
- $1 \leq a_i \leq 10^{18}$
- $1 \leq b_i \leq 10^9$
- $0 \leq |c_i| \leq 10^9$
- 保证存在方案能在 10^9 天内完成任务。

思考一下（10 分钟）

这题并不难。

二分答案

固定正整数 L ，能否在 L 天内完成任务？

观察

- 一个点上越早种上树，到第 L 天结束时长得越高。
- 对每个点都可以算出一个种树的**截止日期**，必须在这天或更早种树。

模型：任务排期

一个**有根树**，每个点代表一个任务。每个任务都有一个完成的截止日期。

做每个任务都需要一天。每天只能做一个任务。

必须在任务 i 的祖先节点都完成后才能做任务 i 。

判断是否每个任务都能在截止日期前完成。

贪心策略

考虑截止时间最晚的叶子任务，最后做它们，若有多个这样的任务，顺序任意。

这对吗？

代码

```
vector<int> g[maxn];
int n;
int ddl[maxn];
int parent[maxn];
long long a[maxn], b[maxn], c[maxn];

using i128 = __int128;
i128 calc(int i, int s, int t) {
    if (c[i] >= 0 || b[i] + c[i] * t > 0) // 等差数列
        return (i128) (b[i] + s * c[i] + b[i] + t * c[i]) * (t - s + 1) / 2;
    int t0 = (b[i] - 1) / -c[i];
    if (s > t0)
        return t - s + 1;
    // [s..t0], [t0+1, t]
    return (i128) (b[i] + s * c[i] + b[i] + t0 * c[i]) * (t0 - s + 1) / 2 + (t - t0);
}

void dfs(int u, int p) {
    parent[u] = p;
    for (int v : g[u])
        if (v != p)
            dfs(v, u);
}

int main() {
    // ... 输入
    dfs(1, 0);
    cout << binary_search(check, 1e9, n - 1) << '\n';
}
```

```
bool check(int x) {
    for (int i = 1; i <= n; i++) {
        // 计算 ddl[i]
        int l = 0, r = x + 1;
        while (l + 1 < r) {
            int mid = (l + r) / 2;
            if (calc(i, mid, x) >= a[i])
                l = mid;
            else
                r = mid;
        }
        if (l == 0) return false;
        ddl[i] = l;
    }
    priority_queue<pair<int, int>> q;
    vector<int> deg(n + 1);
    for (int i = 2; i <= n; i++)
        deg[parent[i]]++;
    for (int i = 1; i <= n; i++)
        if (deg[i] == 0)
            q.push({ddl[i], i});

    int t = x;
    while (!q.empty()) {
        auto p = q.top();
        q.pop();
        if (t == 0) return false;
        t = min(t, p.first) - 1;
        int pa = parent[p.second];
        if (--deg[pa] == 0) {
            q.push({ddl[pa], pa});
        }
    }
    return true;
}
```

习题：如何赢得选举

一场选举已投完票，正在计票。有 N 个候选人，编号 $1, 2, \dots, N$ 。

共收到 K 张选票，其中一些已经计过。目前，候选人 i 已经得到 A_i 张选票。

计票完成后，候选人 i 当选当且仅当得票比他多的候选人少于 M 个。可能有多人当选。

对每个候选人，求

- 剩下的选票，他至少再得到多少张就一定会当选。若他不可能当选，输出 -1 。

限制

- $1 \leq M \leq N \leq 2 \times 10^5$
- $1 \leq K \leq 10^{12}$
- $0 \leq A_i \leq 10^{12}$
- $\sum_{i=1}^N A_i \leq K$

分析

若 $M = N$ ，每个人都必定当选。设 $M < N$ 。

令 $R = K - \sum_{i=1}^N A_i$ ，即还没开的选票数量。

考虑二分答案。

若候选人 i 再得到 x 张选票是否一定当选？

- 先看 $A_1, \dots, A_N$ 中大于 $A_i + x$ 的数有几个，设有 p 个。若 $p \geq M$ ，否。
- 若 $p < M$ ，考虑除了那 p 个人和 i 以外的 $N - p - 1$ 个人；看剩下的选票够不够使得这些人中目前得票最多的那 $M - p$ 个人的最终票数都大于 $A_i + x$ 。
- 设这 $M - p$ 个人已得票的总数是 s 。若 $s + R - x \geq (A_i + x + 1)(M - p)$ ，候选人 i 不一定当选；否则 i 一定当选。
- p 可用二分查找得到。 s 可由前缀和得到。
- 可在 $O(\log N)$ 时间内回答这个问题。

代码

```
int main() {
    int n, m; long long k; cin >> n >> m >> k;
    vector<long long> a(n);
    for (int i = 0; i < n; i++) { cin >> a[i]; k -= a[i]; }
    if (m == n) {
        for (int i = 0; i < n; i++) cout << 0 << ' ';
        return 0;
    }
    auto b = a; sort(b.begin(), b.end());
    vector<long long> s(n + 1); //前缀和
    for (int i = 0; i < n; i++)
        s[i + 1] = s[i] + b[i];

    for (long long x : a) {
        auto check = [&](long long t) {
            int i = upper_bound(b.begin(), b.end(), x + t) - b.begin();
            if (n - i >= m) return false;
            int cnt = m - (n - i);
            long long have;
            if (b[n - m - 1] >= x) have = s[i] - s[i - cnt];
            else have = s[i] - s[i - cnt - 1] - x;
            return have + k - t < cnt * (x + t + 1);
        };
        long long t = binary_search(check, k + 1, -1);
        if (t == k + 1) t = -1;
        cout << t << ' ';
    }
}
```

Bonus

这题有更快的解法。

习题：磨牙

小明有 $2N$ 颗牙： N 颗上牙和 N 颗下牙。

从左到右数，第 i 颗上牙的长度是 U_i ，第 i 颗下牙的长度是 D_i 。

若下列两条件成立，则称他的牙齿**咬合良好**：

- 存在一个整数 H 使得对每个 $i = 1, 2, \dots, N$ 都有 $U_i + D_i = H$ 。
- 对每个整数 $i = 1, 2, \dots, N - 1$ 都有 $|U_i - U_{i+1}| \leq X$ 。

小明可进行下述操作任意多次

- 花 1 元钱使用磨牙机，使他的某颗长度大于等于 1 的牙的长度减少 1。

小明要让他的牙咬合良好最少要花多少钱？

限制

- $2 \leq N \leq 2 \times 10^5$
- $1 \leq U_i, D_i, X \leq 10^9$

习题：平方价格

有 N 种产品，每种有 10^{100} 个。

每种产品你可以买任意非负整数个。买 k 个第 i 种产品要花 $k^2 P_i$ 元。

若你有 M 元，最多可以买多少个产品？

限制

- $1 \leq N \leq 2 \times 10^5$
- $1 \leq M \leq 10^{18}$
- $1 \leq P_i \leq 2 \times 10^9$
- N, M, P_i 都是整数。